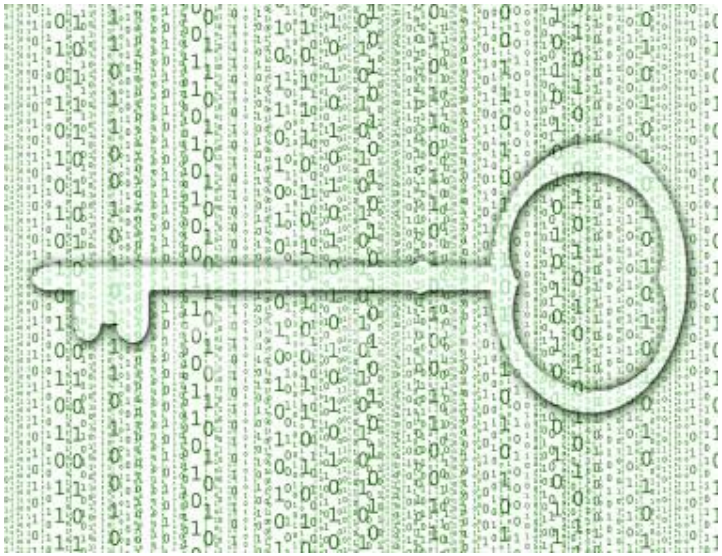
Cryptography





Dec 29

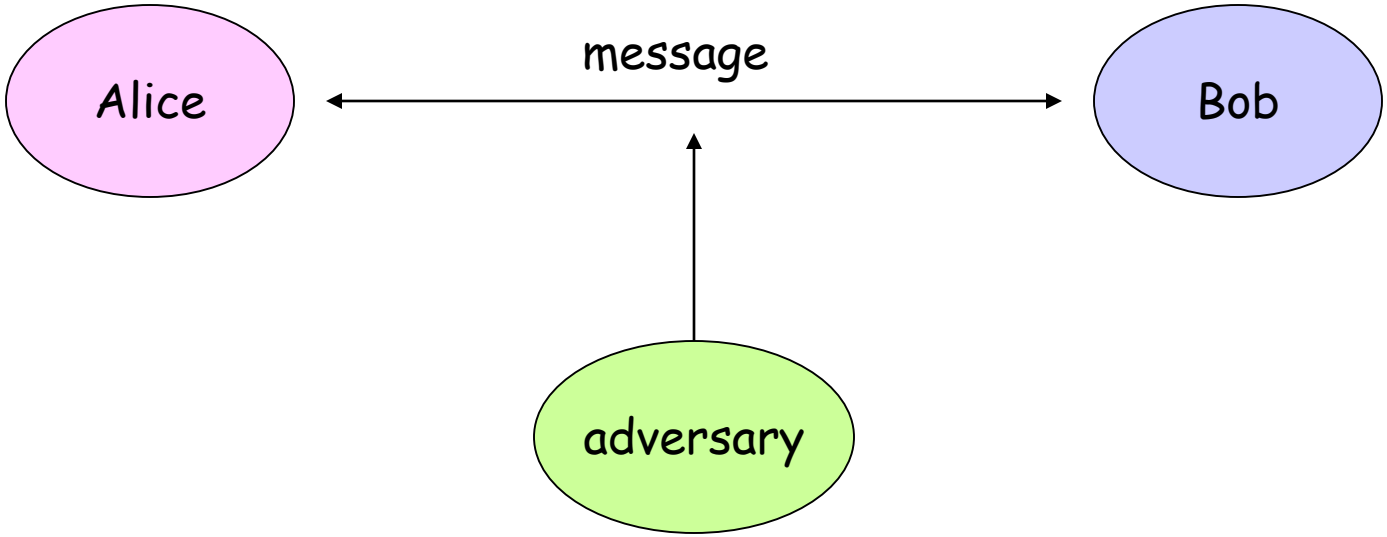
This Lecture

In this last lecture for number theory, we will see probably the most important application of number theory in computer science – the design of cryptosystem.

- Introduction to cryptograph
- "Turing code"
- Public key cryptography
- RSA cryptosystem

Cryptography

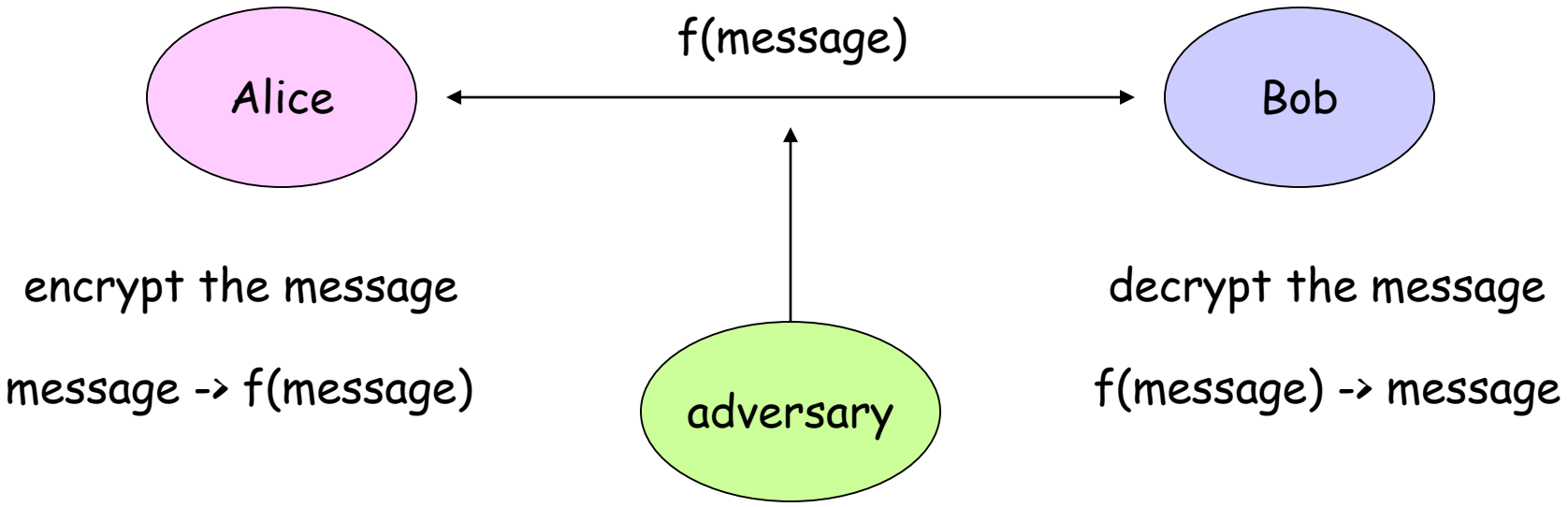
Cryptography is the study of methods for sending and receiving secret messages.



Goal: Even though an adversary can listen to your conversation, the adversary can not learn what the message was.

Cryptography

Goal: Even though an adversary can listen to your conversation, the adversary can not learn what the message was.

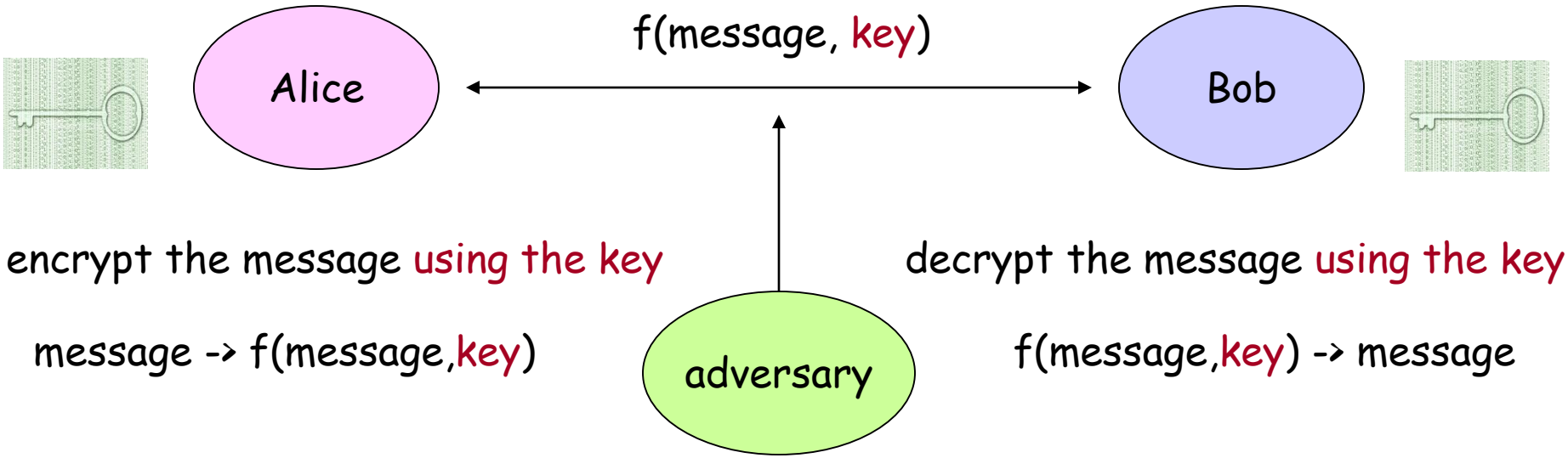


But the adversary has no clue how to obtain message from f(message)

A difficult goal!

Key

Goal: Even though an adversary can listen to your conversation, the adversary can not learn what the message was.



But the adversary can not decrypt f(message,key) without the key

Use number theory!

This Lecture

- Introduction to cryptograph
- "Turing code"
- Public key cryptography
- RSA cryptosystem

Turing's Code (Version 1.0)

The first step is to translate a message into a number

"v i c t o r y"

-> 22 09 03 20 15 18 25

Beforehand The sender and receiver agree on a secret key, which is a large number k.

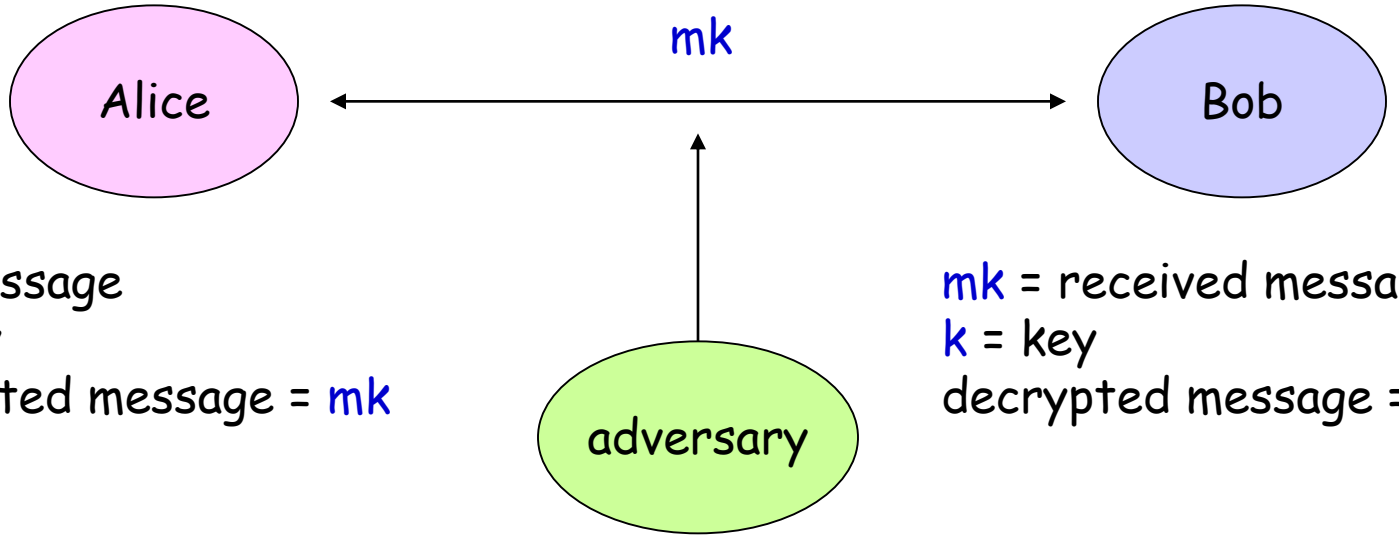
Encryption The sender encrypts the message m by computing:

$m^* = m \cdot k$

Decryption The receiver decrypts m by computing:

$m^*/k = m \cdot k/k = m$

Turing's Code (Version 1.0)



m = message
k = key
encrypted message = mk

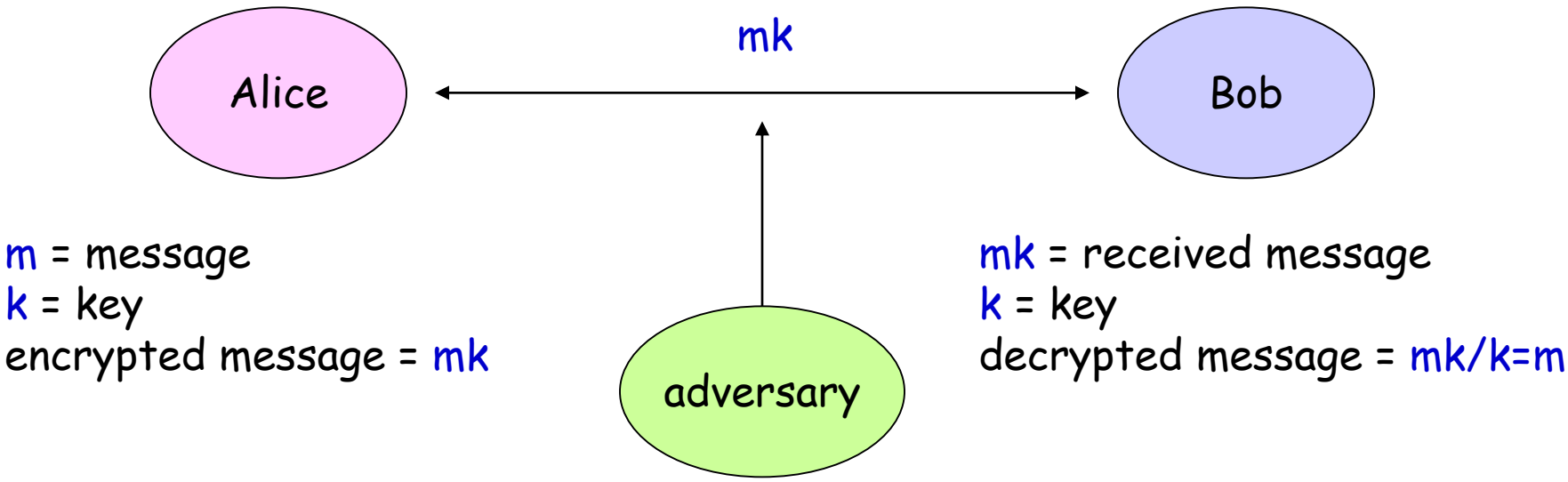
mk = received message
k = key
decrypted message = mk/k=m

Why the adversary cannot figure out m?

The adversary doesn't have the key k,
and so can only factor mk to figure out m,
but factoring is a difficult task to do.



Turing's Code (Version 1.0)



So why don't we use this Turing's code today?

Major flaw: if you use the same key to send two messages m and m',
then from mk and m'k,
we can use gcd(mk,m'k) to figure out k,
and then decrypt every message.

Turing's Code (Version 2.0)

Beforehand The sender and receiver agree on a large prime p, which may be made public. (This will be the modulus for all our arithmetic.) They also agree on a secret key k in $\{1, 2, \ldots, p-1\}$.

Encryption The message m can be any integer in the set $\{0, 1, 2, \ldots, p-1\}$. The sender encrypts the message m to produce m^* by computing:

$$m^* = mk \bmod p$$

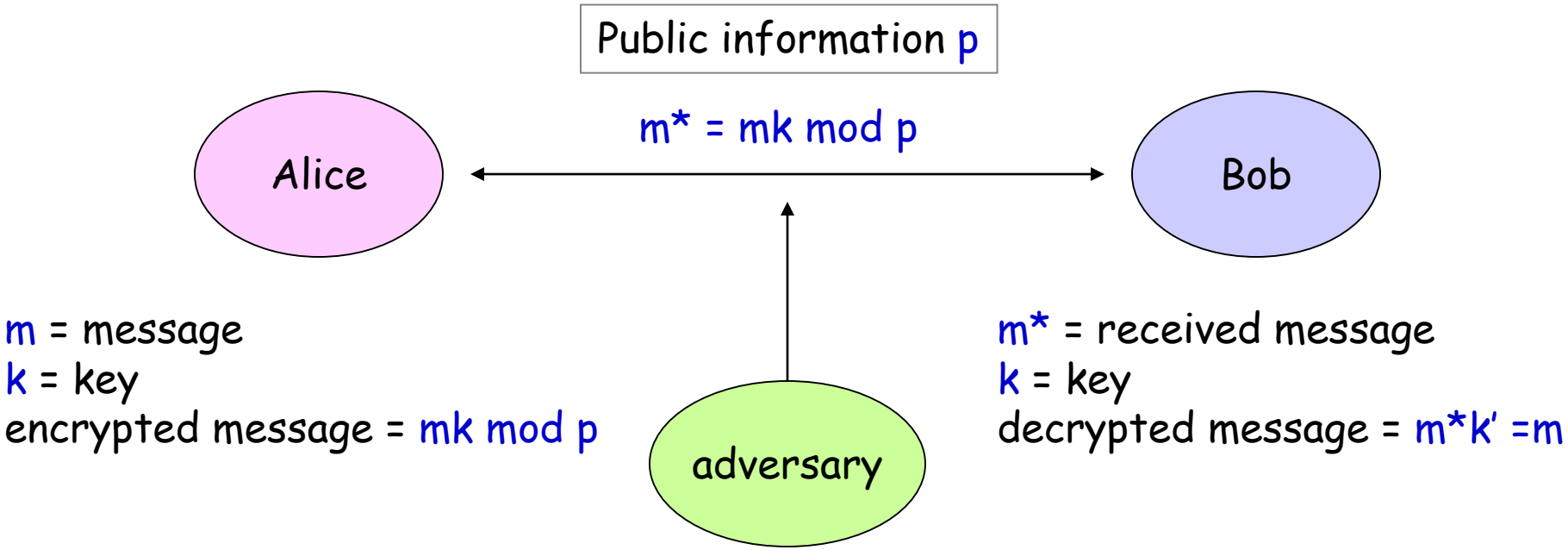
Decryption Let k' be the multiplicative inverse of k under modulo p.

$$m^* \equiv mk \pmod{p}$$

$$m^*k' \equiv m \pmod{p}$$

$$m^*k' = m$$

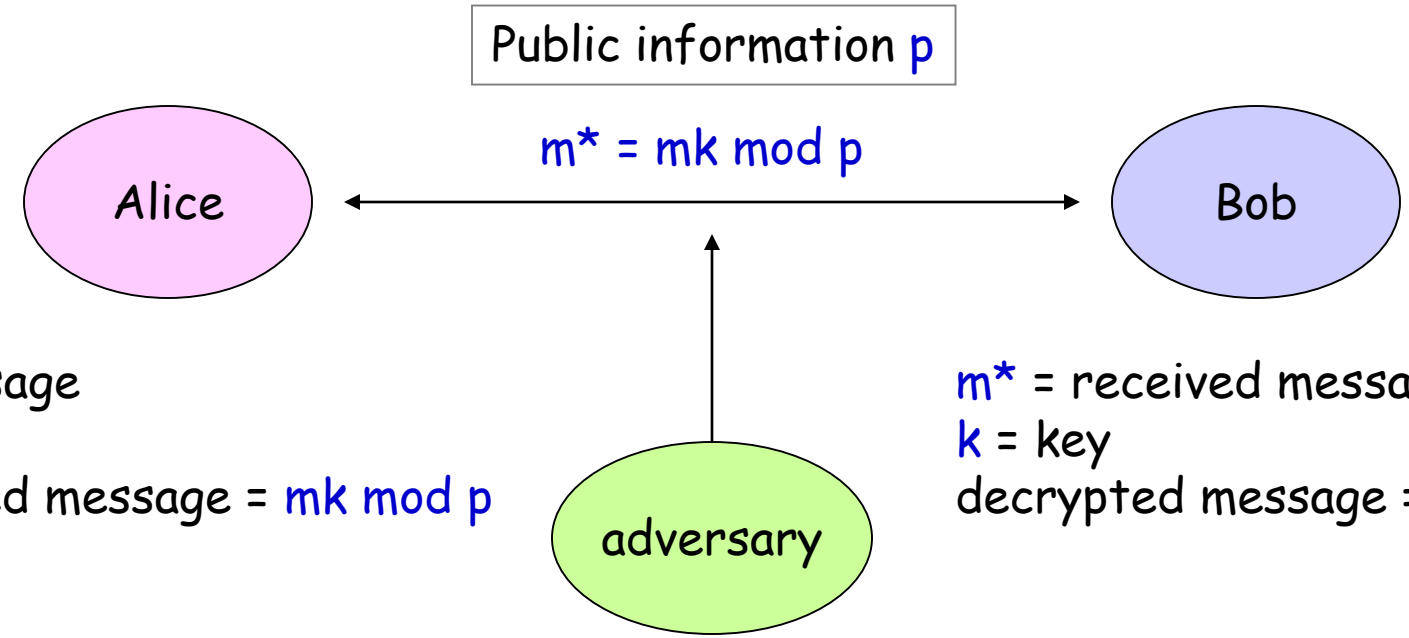
Turing's Code (Version 2.0)



Why the adversary cannot figure out m?

Many m and k can produce m* as output,
just impossible to determine m without k.

Turing's Code (Version 2.0)



m = message
k = key
encrypted message = mk mod p

m* = received message
k = key
decrypted message = m*k' =m

So why don't we use this Turing's code today?

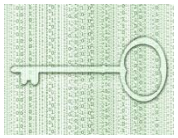
plain-text attack

If the adversary somehow knows m,
then first compute m' := multiplicative inverse of m
$m^* \equiv mk \pmod p$
$m^*m' \equiv k \pmod p$
So the adversary can figure out k.

This Lecture

- Introduction to cryptograph
- "Turing code"
- Public key cryptography
- RSA cryptosystem

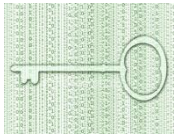
Private Key Cryptosystem



Alice

f(message, key)

Bob



encrypt the message using the key

message -> f(message,key)

adversary

decrypt the message using the key

f(message,key) -> message

But the adversary can not decrypt f(message,key) without the key

Two parties have to agree on a **secret key**, which may be difficult in practice.

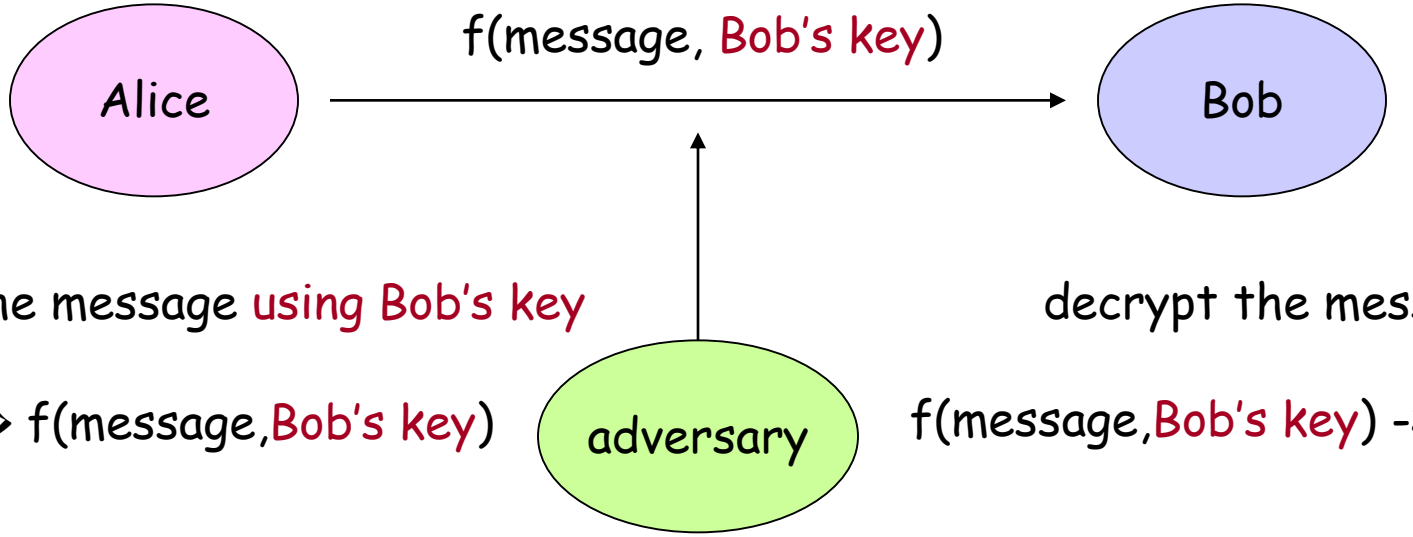
If we buy books from Amazon, we don't need to exchange a secret code.

Why is it secure?

Public Key Cryptosystem

Public information: Key for Alice

Public information: Key for Bob



But the adversary can not decrypt f(message, Bob's key)!

Only Bob can decrypt the message sent to him!

There is no need to have a secret key between Alice and Bob.

How is it possible???

RSA Cryptosystem



RSA are the initials of three Computer Scientists, Ron Rivest, Adi Shamir and Len Adleman, who discovered their algorithm when they were working together at MIT in 1977.

This Lecture

- Introduction to cryptograph
- "Turing code"
- Public key cryptography
- RSA cryptosystem
  - Key generation, encryption, decryption
  - Correctness
  - Secure?
  - Computational issues

Generating Public Key



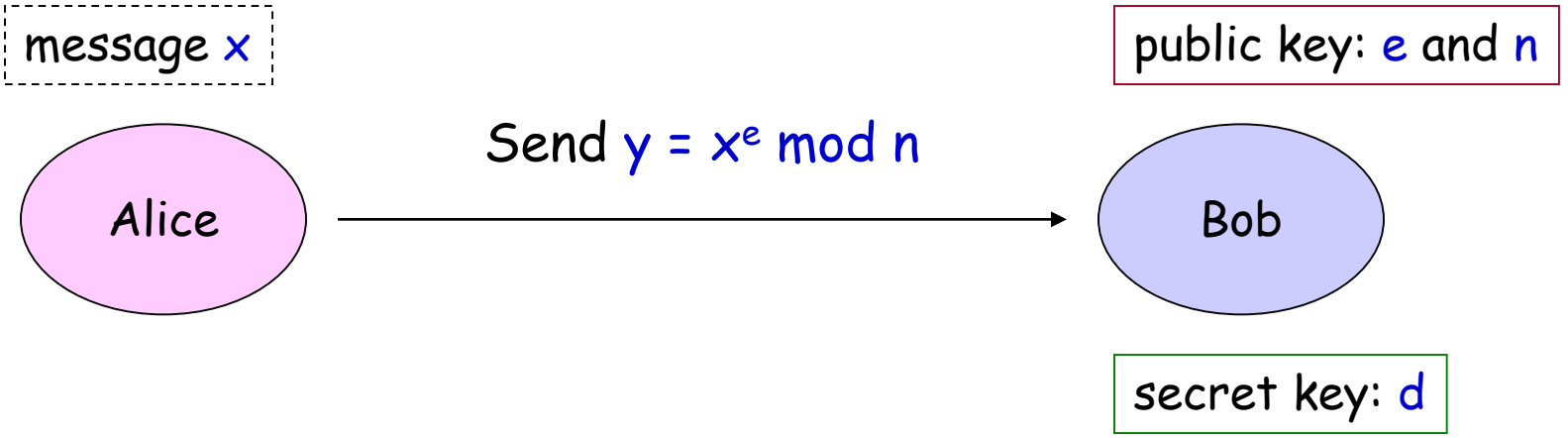
How Bob creates his public keys?

Secret key only known to Bob.

- Choose 2 large prime numbers p and q.
- Set n = pq and T = (p-1)(q-1)
- Choose e ≠1 so that gcd(e,T)=1
- Calculate d so that de = 1 (mod T)
- Publish e and n as public keys
- Keep d as secret key

> 150 digits

Encrypting Message

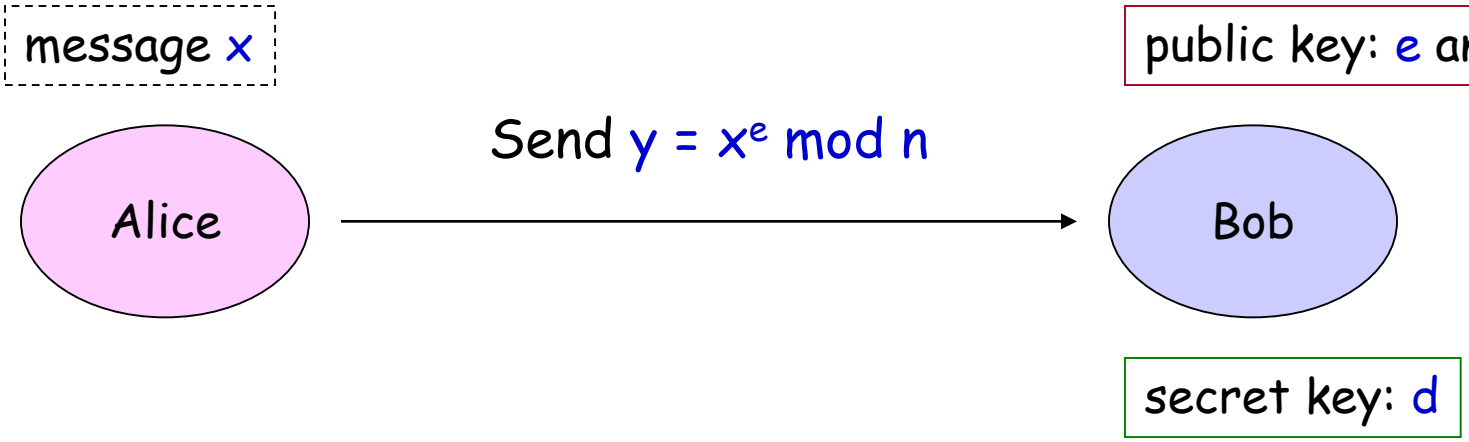


How Alice sends a message to Bob?

- Look at Bob's homepage for e and n.
- Send $y = x^e \bmod n$

Alice does not need to know Bob's secret key to send the message.

Decrypting Message

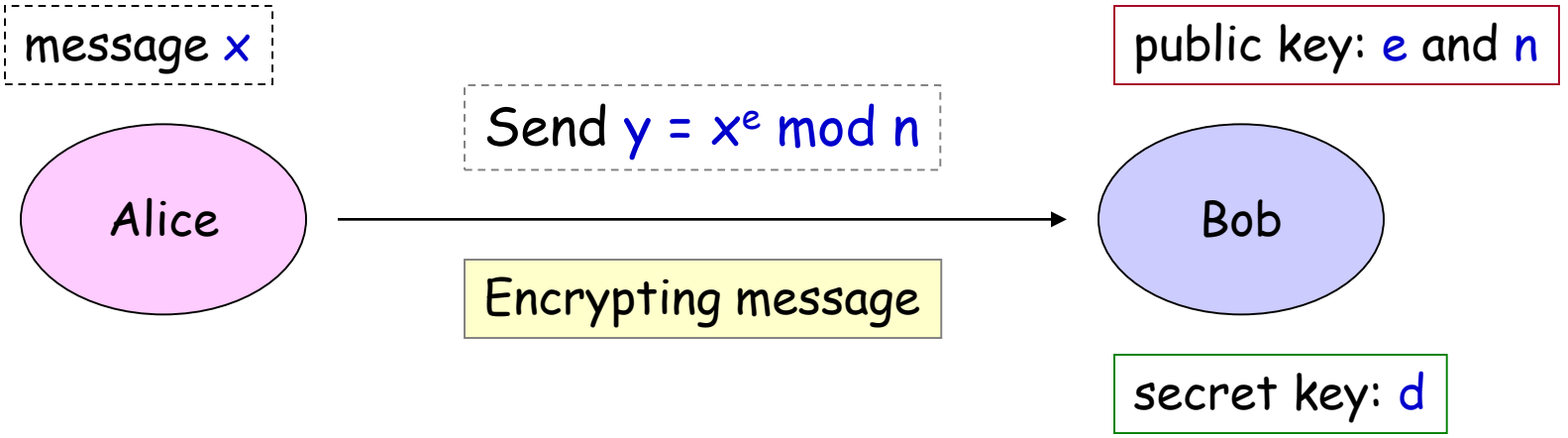


How Bob recover Alice's message?

- Receive $y = x^e \bmod n$
- Compute $z = y^d \bmod n$

Bob uses z is the original message that Alice sent.

RSA Cryptosystem



Key generation

- Choose 2 large prime numbers p and q.
- Set n = pq and T = (p-1)(q-1)
- Choose e ≠1 so that gcd(e,T)=1
- Calculate d so that de = 1 (mod T)
- Publish e and n as public keys
- Keep d as secret key

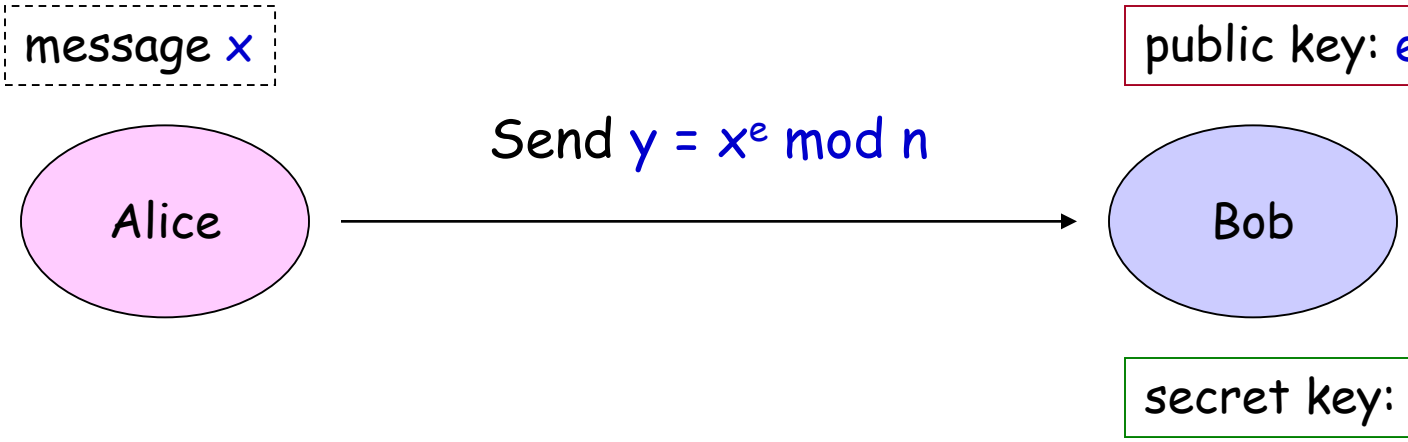
Decrypting message

Compute $z = y^d \bmod n$

This Lecture

- Introduction to cryptograph
- "Turing code"
- Public key cryptography
- RSA cryptosystem
  - Key generation, encryption, decryption
  - Correctness
  - Secure?
  - Computational issues

RSA Cryptosystem



For the RSA cryptosytem to work,
we need to show:

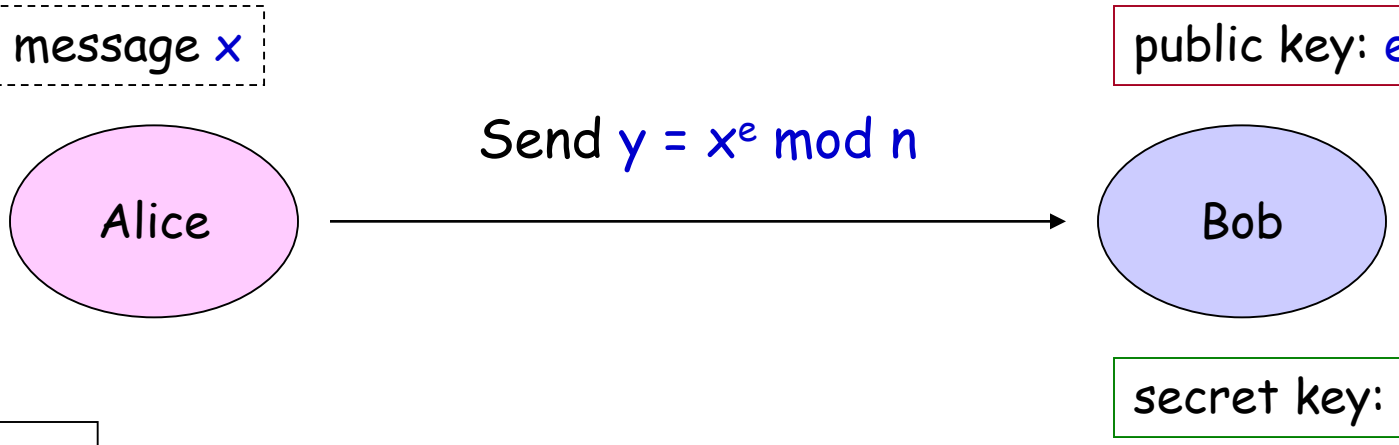
1) $z = x$

2) Without the secret key d,
   we can not compute the original message
   before the sun burns out.

Compute $z = y^d \bmod n$

with additional assumptions…

Correctness



1) z = x

Compute $z = y^d \bmod n$

Note that $z = y^d \bmod n = x^{ed} \bmod n$.

Therefore we need to prove $x = x^{ed} \bmod n$.

(a) $x \bmod p = x^{ed} \bmod p$

(b) $x \bmod q = x^{ed} \bmod q$

(c) $x \bmod n = x^{ed} \bmod n$

p, q prime

n = pq

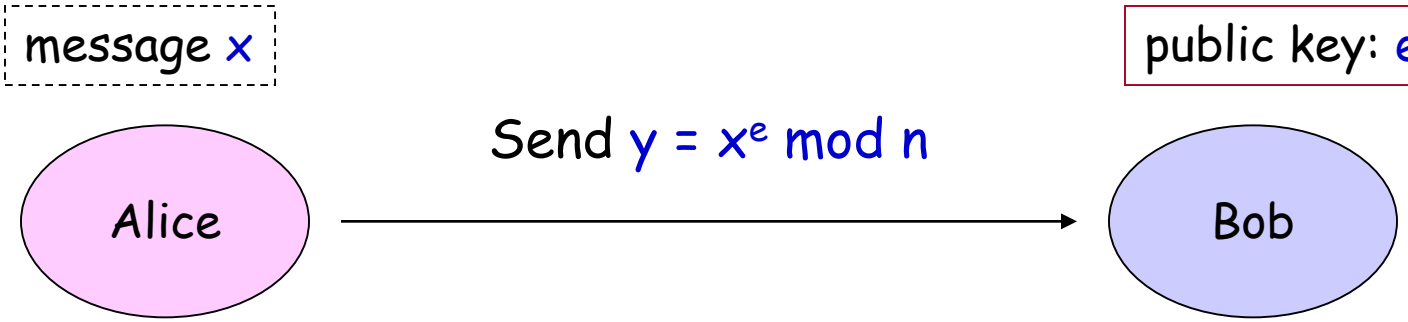
T = (p-1)(q-1)

e s.t. gcd(e,T)=1

de = 1 (mod T)

Therefore, if Alice sends x < n, then Bob can recover correctly.

Correctness



1) $z = x$

(a) $x \bmod p = x^{ed} \bmod p$

Note that $de = 1 + kT = 1 + k(p-1)(q-1)$

Hence, $x^{ed} \bmod p = x^{1+k(p-1)(q-1)} \bmod p$

$= x \cdot x^{k(p-1)(q-1)} \bmod p$

$= x \cdot (x^{k(q-1)})^{(p-1)} \bmod p$

p, q prime

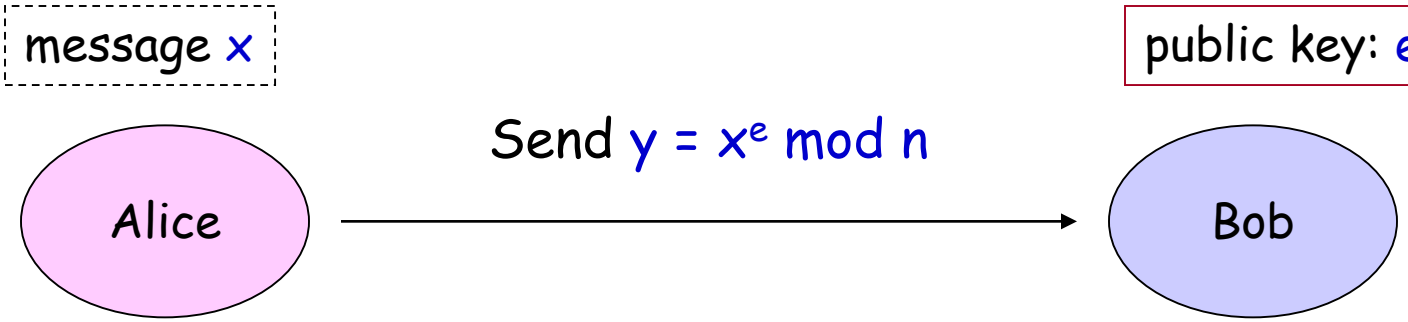
$n = pq$

$T = (p-1)(q-1)$

e s.t. $gcd(e,T)=1$

$de = 1 \pmod T$

Correctness



1) $z = x$

(a) $x \bmod p = x^{ed} \bmod p$

Fermat's little theorem: If $p \nmid a$, then $a^{p-1} \equiv 1 \bmod p$

p, q prime

$n = pq$

$T = (p-1)(q-1)$

e s.t. $\gcd(e,T)=1$

$de = 1 \pmod T$

Hence,

$$x^{ed} \bmod p = x^{1+k(p-1)(q-1)} \bmod p$$

$$= x \cdot x^{k(p-1)(q-1)} \bmod p$$

$$= x \cdot \underbrace{(x^{k(q-1)})}_{a}{}^{(p-1)} \bmod p$$

$$= x \bmod p$$

Correctness

message x

public key: e and n

Send $y = x^e \bmod n$

Alice → Bob

secret key: d

1) $z = x$

(a) $x \bmod p = x^{ed} \bmod p$

Compute $z = y^d \bmod n$

This means $p \mid x^{k(q-1)}$, implying $p \mid x$, since p is prime

p, q prime

$n = pq$

$T = (p-1)(q-1)$

e s.t. $\gcd(e,T)=1$

$de = 1 \pmod T$

Hence, $x^{ed} \bmod p = x^{1+k(p-1)(q-1)} \bmod p$

$= x \cdot x^{k(p-1)(q-1)} \bmod p$

$= x \cdot (x^{k(q-1)})^{(p-1)} \bmod p$

a

What if $p \mid a$?

Since $p \mid x$, we have $x^{ed} \bmod p = x \bmod p = 0$.

Correctness

message x

public key: e and n

Send $y = x^e \bmod n$

Alice → Bob

secret key: d

1) $z = x$

Compute $z = y^d \bmod n$

Note that $z = y^d \bmod n = x^{ed} \bmod n$.

Therefore we need to prove $x = x^{ed} \bmod n$.

(a) $x \bmod p = x^{ed} \bmod p$ ✓

(b) $x \bmod q = x^{ed} \bmod q$ ✓ ← The same proof.

(c) $x \bmod n = x^{ed} \bmod n$ ✓

p, q prime

$n = pq$

$T = (p-1)(q-1)$

e s.t. $\gcd(e,T)=1$

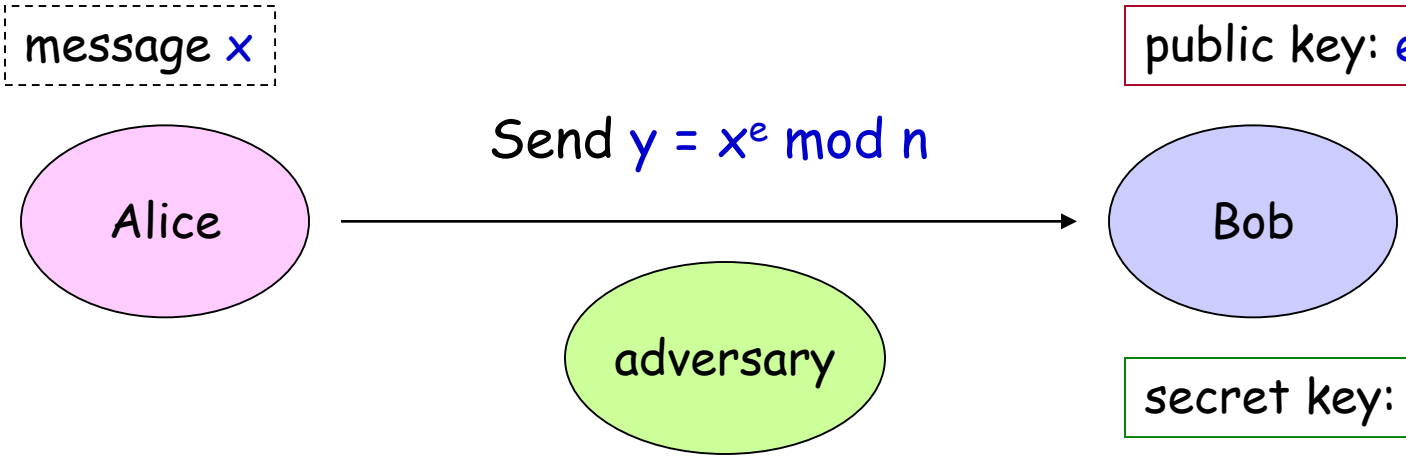
$de = 1 \pmod T$

(c) can be proved directly, also follows from Chinese Remainder theorem.

This Lecture

- Introduction to cryptograph
- "Turing code"
- Public key cryptography
- RSA cryptosystem
  - Key generation, encryption, decryption
  - Correctness
  - Secure?
  - Computational issues

Why is this Secure?



2) Without the secret key d,
we can not compute the original message
before the sun burns out.

Compute $z = y^d \bmod n$

p, q prime
n = pq
T = (p-1)(q-1)
e s.t. gcd(e,T)=1
de = 1 (mod T)

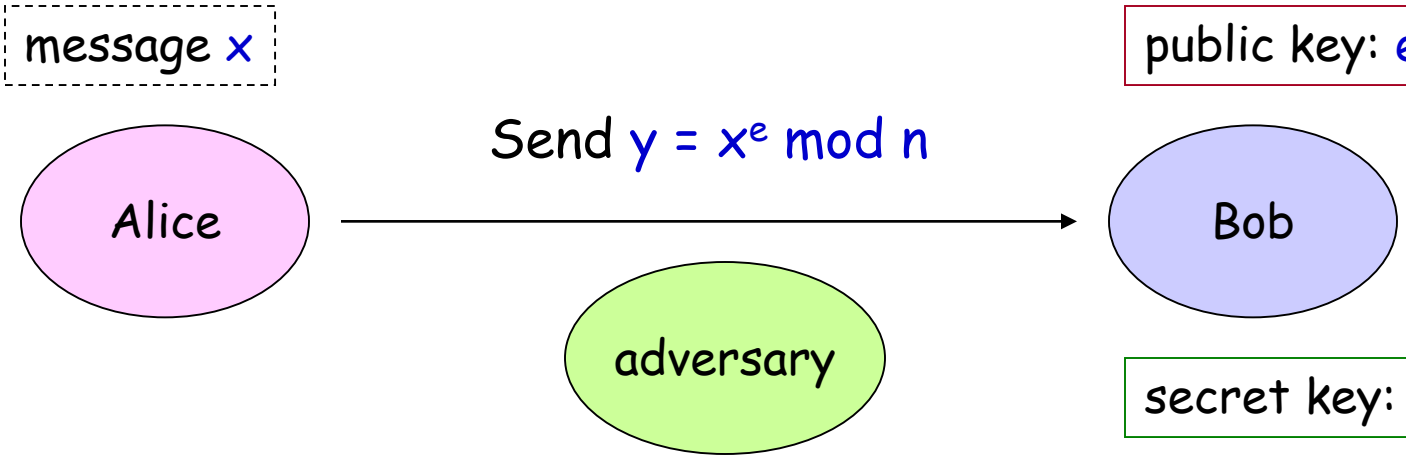
Method 1:

From $y=x^e \bmod n$, don't know how to compute x.

Thus not possible to work backward.

It is an example of an "one-way" function.

Why is this Secure?



2) Without the secret key d,
we can not compute the original message
before the sun burns out.

Compute $z = y^d \bmod n$

Method 2:

Factor n = pq. Compute secrete key d.

Then decrypt everything!

No one knows an efficient way to do factoring.

p, q prime

n = pq

T = (p-1)(q-1)

e s.t. gcd(e,T)=1

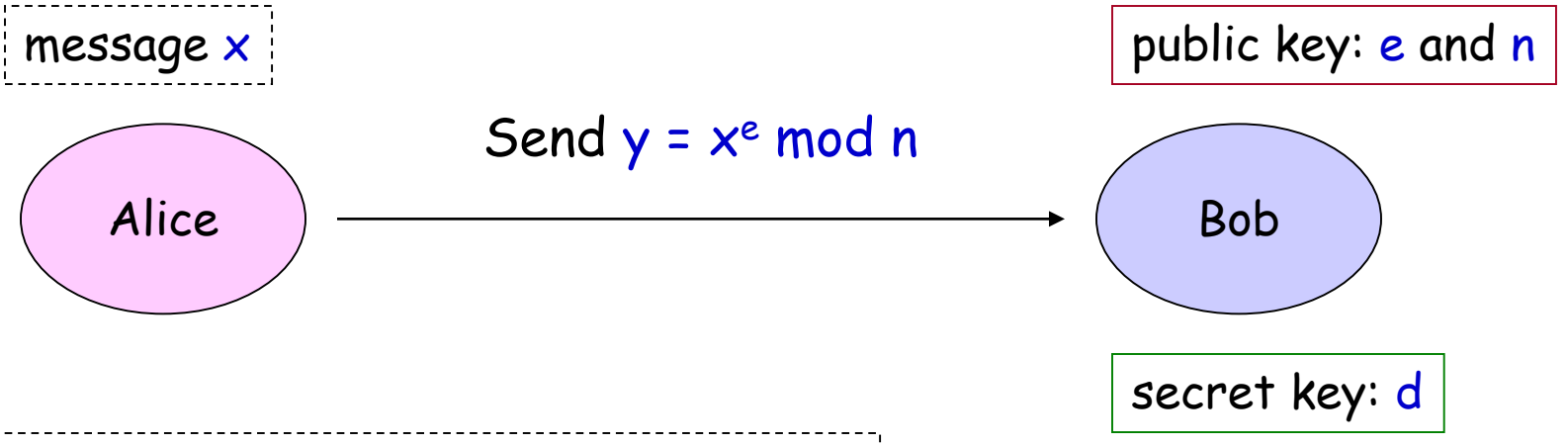
de = 1 (mod T)

The security is based on assumptions that some computational problems are hard.

This Lecture

- Introduction to cryptograph
- "Turing code"
- Public key cryptography
- RSA cryptosystem
  - Key generation, encryption, decryption
  - Correctness
  - Secure?
  - Computational issues

RSA Example



Then Alice sends the encrypted message.

Compute $z = y^d \bmod n$

$x=33$ $y = 33^{23} \bmod 55$

$y = 842986495178819225397387346633991 37 \bmod 55$

How to compute it efficiently?

p=5 q=11
n = 55
T = 40
e = 7
d = 23

→

p, q prime
n = pq
T = (p-1)(q-1)
e s.t. gcd(e,T)=1
de = 1 (mod T)

First Bob generated his keys.

Exponentiation

To compute exponentiation mod n

$144^4 \bmod 713$

$= 144 * 144 * 144 * 144 \bmod 713$

$= 20736 * 144 * 144 \bmod 713$

$= 59 * 144 * 144 \bmod 713$

$= 8496 * 144 \bmod 713$

$= 653 * 144 \bmod 713$

$= 94032 \bmod 713$

$= 629 \bmod 713$

This still takes too long when the exponent is large.

$20736 * 20736 \bmod 713$

$= 59 * 59 \bmod 713$

$= 3481 \bmod 713$

$= 629 \bmod 713$

This is much more efficient.

Repeated Squaring

Note that 50 = 32 + 16 + 2

$144^{50} \bmod 713$

$= 144^{32}\ 144^{16}\ 144^{2} \bmod 713$

$= 648 \cdot 485 \cdot 59 \bmod 713$

$= 242$

$144^{2} \bmod 713 = 59$

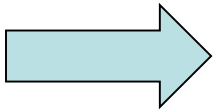
$144^{4} \bmod 713$
$= 144^{2} \cdot 144^{2} \bmod 713$
$= 59 \cdot 59 \bmod 713$
$= 629$

$144^{8} \bmod 713$
$= 144^{4} \cdot 144^{4} \bmod 713$
$= 629 \cdot 629 \bmod 713$
$= 639$

$144^{16} \bmod 713$
$= 144^{8} \cdot 144^{8} \bmod 713$
$= 639 \cdot 639 \bmod 713$
$= 485$

$144^{32} \bmod 713$
$= 144^{16} \cdot 144^{16} \bmod 713$
$= 485 \cdot 485 \bmod 713$
$= 648$

Generating Public Key



- Choose 2 large prime numbers p and q.
- Set $n = pq$ and $T = (p-1)(q-1)$
- Choose $e \neq 1$ so that $\gcd(e,T)=1$
- Calculate d so that $de = 1 \pmod{T}$
- Publish e and n as public keys
- Keep d as secret key

How to choose large prime numbers efficiently?

Given a large number, how to check whether it is prime efficiently?

Primality Testing

Given a large integer n, determine quickly whether n is prime

First test: for $i = 1,\ldots,\sqrt{n}$, check if i divides n.

We are talking about n with 150 digits.
This simply takes too long (2^{150} steps, sun will burn out).

We are looking for an exponential improvement
(instead of n, we can only afford roughly $\log(n)$ steps),
like we did in the extended GCD algorithm.

Need some number theory!

Primality Testing

Theorem: n is a prime if and only if

$$(n-1)! \equiv -1 \pmod{n}$$

It doesn't seem to help,
since we don't know how to compute (n-1)! mod n quickly
(in roughly log(n) steps).

Primality Testing

Theorem: If n is prime & a not a multiple of n

$$1 \equiv a^{n-1} \pmod{n}$$

Contrapositive. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Example. Show that 1763 is composite (not a prime number).

Let $a=2$, $n=1763$.

$2^{1762} \pmod{1763} = 142 \neq 1$

Therefore, it is composite by (the contrapositive of) Fermat's little theorem.

Primality Testing

Contrapositive. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Example. Show that 1387 is composite (not a prime number).

Let $a=2$, $n=1387$.

$2^{1386} \pmod{1387} = 1$ can not tell whether n is prime or not.

Try $a=3$

$3^{1386} \pmod{1387} = 1238 \neq 1$ this shows n is composite.

Primality Testing

Contrapositive. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n,

then n is not a prime number.

"Fermat" test

Given n, choose $a < n$

Compute $a^{n-1} \pmod{n}$

If $a^{n-1} \pmod{n} \neq 1$

conclude that n is a composite number

If $a^{n-1} \pmod{n} = 1$

try another a

Each test takes about $\log(n)$ steps.

It depends on how many a that we need to try…

Primality Testing

Contrapositive. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

"Fermat" test

Given n, choose a < n

Compute $a^{n-1} \pmod{n}$

If $a^{n-1} \pmod{n} \neq 1$

conclude that n is a composite number

If $a^{n-1} \pmod{n} = 1$

try another a

Unfortunately, there exists n which is composite,

but $a^{n-1} \pmod{n} = 1$ for every a!

These are called Carmichael numbers (e.g. 561, 1105, 1729, etc…)

Primality Testing

Contrapositive 1. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Lemma. If n is a prime number, $x^2 \equiv 1 \pmod{n}$ if and only if $x \equiv 1 \pmod{n}$ or $x \equiv -1 \pmod{n}$

Contrapositive 2. If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$ and $x \not\equiv -1 \pmod{n}$ then n is a composite number.

Example

Note that it is $(2^{693})^2$

For $n=1387$ and $a=2$, Fermat's test fails because $2^{1386} \equiv 1 \pmod{1387}$.

However $2^{693} \equiv 512 \pmod{1387} \not\equiv 1 \pmod{1387}$

By **contrapositive 2**, we can conclude that 1387 is a composite number.

Primality Testing

Contrapositive 1. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Contrapositive 2. If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$ and $x \not\equiv -1 \pmod{n}$ then n is a composite number.

Strong primality test

Let $n-1 = 2^k d$ Pick an a.

Compute $a^{2^k d} \pmod{n}$, $a^{2^{k-1} d} \pmod{n}$, $a^{2^{k-2} d} \pmod{n}$,…, $a^d \pmod{n}$

$\neq 1$

Composite by **contrapositive 1.**

Primality Testing

Contrapositive 1. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Contrapositive 2. If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$ and $x \not\equiv -1 \pmod{n}$ then n is a composite number.

Strong primality test

Let $n-1 = 2^k d$ Pick an a.

Compute $a^{2^k d} \pmod{n}$, $a^{2^{k-1} d} \pmod{n}$, $a^{2^{k-2} d} \pmod{n}$,…, $a^d \pmod{n}$

$=1$ $\quad\quad$ $\neq 1$ & $\neq -1$

Composite by **contrapositive 2.**

Primality Testing

Contrapositive 1. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Contrapositive 2. If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$ and $x \not\equiv -1 \pmod{n}$ then n is a composite number.

Strong primality test

Let $n-1 = 2^k d$ Pick an a.

Compute $a^{2^k d} \pmod{n}$, $a^{2^{k-1} d} \pmod{n}$, $a^{2^{k-2} d} \pmod{n}$,…, $a^d \pmod{n}$

$=1$ $=1$

Continue to go backward and check

Primality Testing

Contrapositive 1. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Contrapositive 2. If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$ and $x \not\equiv -1 \pmod{n}$ then n is a composite number.

Strong primality test

Let $n-1 = 2^k d$ Pick an a.

Compute $a^{2^k d} \pmod{n}$, $a^{2^{k-1} d} \pmod{n}$, $a^{2^{k-2} d} \pmod{n}$,…, $a^d \pmod{n}$

$=1$ $=1$ $=-1$

End the test and say it is a "probable" prime.

Primality Testing

Contrapositive 1. If $1 \not\equiv a^{n-1} \pmod{n}$ and a is not a multiple of n, then n is not a prime number.

Contrapositive 2. If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$ and $x \not\equiv -1 \pmod{n}$ then n is a composite number.

Strong primality test

Let $n-1 = 2^k d$ Pick an a.

Compute $a^{2^k d} \pmod{n}$, $a^{2^{k-1} d} \pmod{n}$, $a^{2^{k-2} d} \pmod{n}$,…, $a^d \pmod{n}$

$=1$ $=1$ $=1$ $=1$ $=1$

End the test and say it is a "probable" prime.

Primality Testing

Strong primality test

Given n, pick an a

Let $n' = n-1$ (so n' is an even number)

If $a^{n'} \pmod n \neq 1$,

then stop and say "n is composite".

$n' := n'/2$;

While n' is an integer do

If $a^{n'} \pmod n = -1$,

then stop and say "n is a probable prime".

If $a^{n'} \pmod n \neq 1$,

then stop and say "n is composite".

$n' := n'/2$;

Stop and say "n is a probable prime".

Primality Testing

For a particular a, the strong primality test takes "about" log(n) steps.

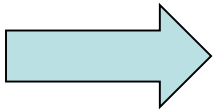
But again, there exists n which is composite but pass the test…

Theorem: if n is composite, for more than half of a < n,
the strong primality test will say n is composite!

So, given a composite n, if we pick a random a,
the strong primality test will be incorrect with probability <= 1/2.
Thus, if we repeat the procedure for 10000 times,
then the probability that the strong primality test is still incorrect
is very small (e.g. much smaller than our computer will suddenly crash).

This is the most efficient method used in practice!

Generating Public Key



- Choose 2 large prime numbers p and q.
- Set $n = pq$ and $T = (p-1)(q-1)$

Similar idea 

- Choose $e \neq 1$ so that $\gcd(e,T)=1$
- Calculate d so that $de = 1 \pmod{T}$
- Publish e and n as public keys
- Keep d as secret key

Prime number theorem: From 1 to n, there are roughly $n/\log(n)$ prime numbers.

How to choose large prime numbers efficiently?

Pick a random large number,
do the (randomized) strong primality tests,
until we find a prime!

Remarks

- We have derived everything from basic principle.

- RSA cryptosystem is one of the most important achievements in compute science. (The researchers won the Turing award for their contribution.)

- Number theory is also very useful in coding theory (e.g. compression).

- Mathematics is very important in computer science.

More Remarks

Theorem: if n is composite, for more than half of $a < n$, the strong primality test will say n is composite!

The proof uses Chinese Remainder theorem and some elementary number theory. (Introduction to Algorithms, MIT press)

Conjecture: It is enough to try a to up to roughly $\log(n)$.

Theroem (Primes is in P, 2004)
There is an efficient and deterministic primality test.

Major Open Problem:
Is there an efficient algorithm to compute the prime factorization?